MINOR PROJECT 05

Docker + Jenkins Automation Project

A small CI pipeline that retrieves source from GitHub, builds a Docker image, and runs the application container from Jenkins.

GitHub repository	Jenkins Build Now	Docker build	Running app
Source code + Jenkinsfile	Checkout stage	Image: jenkins-demo-app	localhost:3000

Project objective

Automate three manual steps with a Jenkins Declarative Pipeline: checkout application code from SCM, build a Docker image, and run a container. The coursework lab used Windows with WSL 2 (Ubuntu), Docker Desktop, and Jenkins running as a Docker container. A job was started manually with **Build Now**.

Tools and concepts

Tool	Use in this project
Jenkins	Pipeline orchestration and stage reporting.
Docker	Package the Node.js app into an image and start a container.
GitHub SCM	Store app.js, Dockerfile, and Jenkinsfile; Jenkins checks out the repository.
WSL 2 + Docker Desktop	Provide an Ubuntu terminal and local Docker engine from a Windows development environment.

Outcome reported in the coursework

The report records a successful run where the Checkout, Build Docker Image, and Run Container stages passed, and the application displayed “Hello from Jenkins + Docker automation!” at **http://localhost:3000**.

Implementation overview

Pipeline stages

Stage	Action	Purpose
Checkout	Check out the configured SCM revision.	Get source code and Jenkinsfile from the repository.
Build Docker Image	Build the image from the project Dockerfile.	Create a reproducible application image.
Run Container	Remove the previous demo container if it exists, then start the new one on port 3000.	Make the app available locally for verification.

Application

The example is a dependency-free Node.js HTTP server. It listens on port 3000 and returns a plain-text success message to incoming requests.

```
const http = require('node:http');
const PORT = Number(process.env.PORT) || 3000;

const server = http.createServer((_req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('Hello from Jenkins + Docker automation!\n');
});

server.listen(PORT, '0.0.0.0');
```

Local verification

```
docker build -t jenkins-demo-app:local ./05-docker-jenkins-automation
docker run --rm -p 3000:3000 jenkins-demo-app:local
# Open http://localhost:3000
```

Jenkins job configuration for the portfolio monorepo

Set the job to **Pipeline script from SCM**, configure the portfolio repository URL, set the branch specifier to `*/main`, and set the Script Path to `05-docker-jenkins-automation/Jenkinsfile`. Save the job and click **Build Now**.

Troubleshooting and security

Problems encountered in the original lab

Issue	Resolution recorded in the coursework
Docker CLI missing in Jenkins container	The lab supplied the Docker CLI inside the Jenkins container.
Docker socket permission denied	The lab recreated the container as root so it could access the mounted host socket. This is a high-privilege workaround, not a recommended production default.
SCM checkout failed	The report identified Lightweight checkout behavior and stale workspace data as causes; it records disabling Lightweight checkout and clearing the stale workspace before rebuilding.

Security limitations

Docker socket access: The original lab mounted `/var/run/docker.sock` into the Jenkins container and ran Jenkins as root to bypass socket permission problems. A pipeline with access to the host Docker daemon can gain broad control over that host. Keep this setup isolated and do not use it for untrusted jobs or a public/shared Jenkins service.

Jenkins exposure: Keep the Jenkins UI authenticated and local/isolated for this learning setup. Do not expose it to the public internet without appropriate hardening.

Runtime update: The coursework report used `node:18-alpine`. The Dockerfile in this portfolio package uses `node:24-alpine` and runs the app as a non-root user. Node.js lists v18 as end-of-life, so the lab's original base image should not be reused for new production work. See official Node.js release information.

Next improvements

Add automated tests, a post-deployment HTTP smoke test, webhook-based triggers, pinned Jenkins/image versions, image scanning, and an isolated build agent rather than mounting the host Docker socket into the Jenkins controller.

Source note

This portfolio report is a cleaned technical summary based on the submitted coursework PDF. The original screenshots are not embedded here, to avoid carrying environment-specific desktop and terminal details into the public version. The pipeline source is included separately in the project folder.